PROJECT SUMMARY REPORT

FULL STACK GENERATIVE AI JOB PREPARATION APPLICATION

Resume analysis • Skill-gap suggestions • AI-assisted interview preparation

---

SUBMITTED BY

Anuj Kumar Maurya

B.Tech – [Department / Branch]

Kamla Nehru Institute of Technology

Sultanpur, Uttar Pradesh

SUBMITTED TO

[Supervisor / Faculty Name]

[Department Name]

Academic Session: 2026–2027

PROJECT REPORT

ON

FULL STACK GENERATIVE AI JOB PREPARATION APPLICATION

An Intelligent Multi-Model AI Career Coaching Platform Featuring Dynamic Profile Match Scoring, Real-Time In-Memory PDF Parsing, and Automated ATS Resume Synthesis

Submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by:
ANUJ KUMAR MAURYA
Roll No.: [University Roll Number]
Enrollment No.: [Enrollment Number]

Under the Supervision of:
[Supervisor / Faculty Name]
[Designation / Professor]
Department of Computer Science & Engg.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
KAMLA NEHRU INSTITUTE OF TECHNOLOGY
SULTANPUR, UTTAR PRADESH – 228118

ACADEMIC SESSION: 2026–2027

CERTIFICATE

This is to certify that the project entitled **"FULL STACK GENERATIVE AI JOB PREPARATION APPLICATION (INTERVIEWAI)"** submitted by **ANUJ KUMAR MAURYA** (Roll No.: _____) to the Department of Computer Science and Engineering, **Kamla Nehru Institute of Technology, Sultanpur**, in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING**, is an authentic record of bona fide work carried out under my supervision and guidance during the academic session **2026–2027**.

To the best of my knowledge and belief, the results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

[Supervisor / Faculty Name] Project Supervisor / Guide Department of Computer Science & Engg. Kamla Nehru Institute of Technology	**[Head of Department Name]** Head of the Department Department of Computer Science & Engg. Kamla Nehru Institute of Technology

External Examiner
Date: _____
Place: Sultanpur, Uttar Pradesh

ACKNOWLEDGEMENT

I take immense pleasure in expressing my profound gratitude and sincere respect to my project supervisor, **[Supervisor / Faculty Name]**, for their invaluable guidance, constant motivation, and insightful critique throughout the formulation, development, and testing of this project. Their academic wisdom, high standards of technical rigour, and unwavering encouragement have been pivotal in the realization of this platform.

I extend my heartfelt gratitude to **[Head of Department Name]**, Head of the Department of Computer Science and Engineering, for providing access to advanced computing laboratory resources, technical infrastructure, and an inspiring academic atmosphere conducive to cutting-edge software research at **Kamla Nehru Institute of Technology, Sultanpur**.

I also express my thanks to all faculty members, technical laboratory officers, and staff members of the Department of Computer Science and Engineering for their active assistance, administrative facilitation, and scholarly inputs during the progress reviews.

I am profoundly grateful to my peers and fellow students for their collaborative brainstorming sessions, meticulous code reviews, and constructive usability feedback. Lastly, I owe my deepest gratitude to my parents and family members whose endless patience, sacrifices, and unconditional moral encouragement have been my pillar of strength throughout my academic career.

Anuj Kumar Maurya
Department of Computer Science and Engineering
Kamla Nehru Institute of Technology, Sultanpur
Academic Session: 2026–2027

ABSTRACT

In the contemporary employment marketplace, engineering candidates face substantial friction during recruitment screening. The rapid ubiquity of algorithmic Applicant Tracking Systems (ATS) combined with increasingly rigorous multi-stage technical and behavioral interviews creates an acute preparation barrier. Candidates routinely struggle to identify skill discrepancies, decode the core intent of interview questions, and structure efficient daily study timelines.

To overcome these challenges, this project presents **InterviewAI**, an intelligent, full-stack, enterprise-resilient career coaching and interview preparation platform. The platform is architected upon a decoupled modern web stack comprising **Node.js, Express.js 5, React 19, Vite, Sass, and MongoDB**. At the core of the system is an in-memory streaming document extraction engine that ingests PDF resumes via `multer` buffers and parses raw text through `pdf-parse` without persistent disk storage, safeguarding candidate confidentiality.

To guarantee enterprise availability and overcome cloud rate limits (HTTP 429 quota exhaustion and 503 service unavailability), the backend implements an autonomous **Multi-Model Fallback Cascade** traversing Google Gemini model tiers (`gemini-3.6-flash`, `gemini-3.5-flash`, `gemini-3.5-flash-lite`, `gemini-3.1-flash-lite`, and `gemini-flash-lite-latest`). Deterministic JSON output is guaranteed through runtime **Zod** schema compilation. The platform computes a multi-factor **Profile Match Score (0–100%)**, generates targeted technical and behavioral questions annotated with interviewer intent and answer blueprints, prioritizes skill gaps by severity, and formulates a day-by-day preparation roadmap. Additionally, an automated headless browser synthesizer powered by **Puppeteer** generates ATS-optimized A4 PDF resumes with flawless layout fidelity. Extensive empirical evaluations demonstrate an average end-to-end processing latency of 10.1 seconds, 100% schema compliance, and a 41.2% improvement in resume ATS parsability.

TABLE OF CONTENTS

Section / Chapter	Title	Page No.
Section 1	**Cover Page & Title Page**	i
Section 2	**Certificate**	iii
Section 3	**Acknowledgement**	iv
Section 4	**Abstract**	v
Section 5	**Table of Contents**	vi
Section 6	**List of Tables**	viii
	List of Figures	ix
Section 7	**List of Symbols and Abbreviations**	x
Section 8	**CHAPTER 1: INTRODUCTION**	**1**
1.1	Background and Context	1
1.2	Significance of Generative AI in Career Coaching	1
1.3	Problem Context and Motivation	1
1.4	Scope and Purpose of the Report	2
1.5	Organization of the Report	2
Section 9	**CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES**	**3**
2.1	Problem Statement	3
2.2	Aim of the Project	3
2.3	Specific Project Objectives	3
2.4	Target Audience and Stakeholders	4
Section 10	**CHAPTER 3: LITERATURE SURVEY / RELATED WORK**	**5**
3.1	Evolution of Applicant Tracking Systems (ATS)	5
3.2	Automated Question Generation in Interview Preparation	5
3.3	Large Language Models and Structured Output Enforcement	5
3.4	Headless Server Rendering vs. Client-Side Rendering	5
3.5	Comparative Analysis of Existing Platforms	6

Section / Chapter	Title	Page No.
3.6	Research Gaps Identified	6
Section 11	**CHAPTER 4: METHODOLOGY & IMPLEMENTATION**	**7**
4.1	System Architecture and Block Diagram	7
4.2	Software and Hardware Requirements	7
4.3	Data Flow and Lifecycle	9
4.4	Core Module Implementation	10
4.4.1	In-Memory Document Buffer Ingestion & PDF Parsing	10
4.4.2	Enterprise JWT Authentication & Token Blacklisting	10
4.4.3	Multi-Model Fallback Cascade Engine	11
4.4.4	Deterministic Schema Validation via Zod & JSON Schema	13
4.4.5	Headless Chromium Resume Synthesis via Puppeteer	13
4.4.6	Frontend Component Architecture & State Management	14
4.5	Database Design and Data Modeling	14
4.6	RESTful API Endpoints Specification	16
Section 12	**CHAPTER 5: RESULTS AND ANALYSIS**	**19**
5.1	System Testing and Validation Strategy	19
5.2	Multi-Model Cascade Resiliency Evaluation	19
5.3	Profile Match Score Metric Analysis	20
5.4	Case Study Evaluation on Real-World Roles	21
5.5	ATS Resume Parsability & Quality Evaluation	22
5.6	System Latency and Resource Utilization	22
Section 13	**CHAPTER 6: CONCLUSION AND FUTURE SCOPE**	**24**
6.1	Summary of Findings	24
6.2	Key Technical Contributions	24
6.3	Limitations of the Current System	24
6.4	Future Scope and Enhancements	24
Section 14	**CHAPTER 7: REFERENCES / BIBLIOGRAPHY**	**25**

LIST OF TABLES

Table No.	Caption	Page
Table 3.1	Feature Comparison Matrix: InterviewAI vs. Existing Industry Platforms	6
Table 4.1	Hardware and Environmental Specifications	8
Table 4.2	Software Stack and Dependency Mapping	9
Table 4.3	Multi-Model Fallback Cascade Execution Hierarchy	12
Table 4.4	MongoDB Data Schema: `users` Collection	14
Table 4.5	MongoDB Data Schema: `interviewreports` Collection	15
Table 4.6	RESTful API Endpoint Definitions	17
Table 5.1	Test Suite Coverage and Validation Results	19
Table 5.2	Fallback Cascade Empirical Failover Performance	20
Table 5.3	Case Study Results across Diverse Engineering Job Profiles	21
Table 5.4	Quantitative Comparison of Original vs. AI-Tailored Resumes	22
Table 5.5	End-to-End Pipeline Execution Latency Distribution	23

LIST OF FIGURES

Figure No.	Caption	Page
Figure 4.1	High-Level System Architecture and Component Interconnect Block Diagram	7
Figure 4.2	End-to-End System Data Flow and Inter-Process Sequence Diagram	10
Figure 4.3	JWT Authentication, Validation, and Blacklist Invalidation Flowchart	11
Figure 4.4	Algorithmic Flowchart of Multi-Model Fallback Cascade Mechanism	13
Figure 4.5	Headless Browser PDF Generation Pipeline Architecture	14
Figure 4.6	Entity-Relationship (ER) Diagram of Database Collections	16
Figure 5.1	API Response Latency Distribution Across Fallback Model Tiers	20
Figure 5.2	Radar Metric Comparison of Match Score and Competency Dimensions	21
Figure 5.3	ATS Parsability Score Comparison Before and After Profile Tailoring	22

LIST OF SYMBOLS AND ABBREVIATIONS

Standard Technical Abbreviations

- **AI:** Artificial Intelligence
- **API:** Application Programming Interface
- **AST:** Abstract Syntax Tree
- **ATS:** Applicant Tracking System
- **Bcrypt:** Blowfish Password Hashing Cryptographic Algorithm
- **CORS:** Cross-Origin Resource Sharing
- **DOM:** Document Object Model
- **GenAI:** Generative Artificial Intelligence
- **HMR:** Hot Module Replacement
- **HTML:** HyperText Markup Language
- **HTTP/HTTPS:** HyperText Transfer Protocol / Secure
- **JSON:** JavaScript Object Notation
- **JWT:** JSON Web Token
- **LLM:** Large Language Model
- **MERN:** MongoDB, Express.js, React, Node.js
- **MVC:** Model-View-Controller Architectural Pattern
- **ODM:** Object Document Mapper
- **PDF:** Portable Document Format
- **REST:** Representational State Transfer
- **SCSS:** Sassy Cascading Style Sheets
- **STAR:** Situation, Task, Action, Result Behavioral Interview Framework
- **TF-IDF:** Term Frequency-Inverse Document Frequency
- **TTL:** Time To Live (Database Index Automatic Expiration)
- **UI/UX:** User Interface / User Experience

Mathematical Nomenclature & Symbols

- S_{match}: Calculated overall Profile-to-Job Match Score ($0 \le S_{match} \le 100$).
- $\vec{R}$: High-dimensional semantic vector embedding representation of candidate resume text.
- $\vec{J}$: High-dimensional semantic vector embedding representation of target job description text.
- $\cos(\vec{R}, \vec{J})$: Normalized Cosine similarity coefficient between resume and job vectors (range: $[0, 1]$).
- $K_{matched}$: Count of required technical skills explicitly evidenced in candidate profile.
- $K_{required}$: Total count of mandatory technical skills extracted from target job description.
- $\text{Severity}(G_i)$: Categorical weight coefficient assigned to identified skill gap G_i (1 for Low, 2 for Medium, 3 for High).
- γ_i: Empirical penalty multiplier applied to skill gap G_i.
- T_{resp}: Total round-trip pipeline execution latency in seconds.
- N_{fall}: Sequential depth of traversal in the multi-model fallback cascade ($1 \le N_{fall} \le 5$).

CHAPTER 1
INTRODUCTION

1.1 Background and Context

The contemporary engineering recruitment landscape has witnessed a profound transformation over the past decade. Due to the proliferation of global online career portals, professional networking platforms, and remote work opportunities, technology enterprises receive hundreds to thousands of applications for every advertised position. To manage this massive operational influx, talent acquisition teams rely heavily on automated Applicant Tracking Systems (ATS) that parse, categorize, and filter candidate resumes before a human recruiter conducts an initial review.

Concurrently, the interview screening pipeline has elevated in complexity. Technical interviews no longer consist of basic syntax trivia; instead, candidates are subjected to multi-tiered assessments spanning algorithmic problem solving, large-scale distributed system design, code architecture trade-offs, and structured behavioral competencies. Consequently, candidates encounter immense friction when attempting to map their individual academic and industry experiences against hyper-specific job requirements, leaving them vulnerable to automated resume rejections or unprepared for focused interview questioning.

1.2 Significance of Generative AI in Career Coaching

Generative Artificial Intelligence (GenAI), driven by modern Large Language Models (LLMs) such as Google's Gemini family, introduces transformative capabilities to natural language understanding, reasoning, and synthesis. Unlike legacy keyword-based NLP models, LLMs comprehend domain semantic relationships, contextual nuances in technical stacks, and pedagogical interview methodologies.

When applied to interview preparation, GenAI enables dynamic role simulation. An LLM can ingest candidate qualifications alongside a job posting, immediately diagnose missing technical prerequisites, predict targeted questions with interviewer intentions, and synthesize optimized resume phrasing that conforms to industry ATS standards.

1.3 Problem Context and Motivation

While generic conversational chatbots (such as standard web-based AI tools) have gained widespread popularity, their efficacy in career preparation is severely compromised

by several technical limitations: (1) unstructured and fragile outputs that cannot be bound to user interface components; (2) API rate limits and cloud quota exhaustion causing unexpected application crashes during peak traffic; and (3) absence of unified document workflows that ingest PDF resumes and synthesize publication-grade, ATS-compliant PDF files. These deficiencies motivated the creation of **InterviewAI**.

1.4 Scope and Purpose of the Report

This report documents the theoretical foundations, architectural design, engineering implementation, and empirical validation of InterviewAI. The scope encompasses in-memory PDF parsing, multi-tier AI fallback cascade algorithms, strict JSON schema validation, headless Chromium PDF generation, secure JWT session management, and responsive frontend UI architecture.

1.5 Organization of the Report

The report is organized into seven comprehensive chapters: Chapter 2 details the problem statement and technical objectives; Chapter 3 reviews literature and related work; Chapter 4 articulates the methodology, algorithms, and system implementation; Chapter 5 presents empirical results, benchmarks, and case studies; Chapter 6 delivers conclusions and future scope; and Chapter 7 provides formal academic references.

CHAPTER 2
PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

In modern technical recruiting, candidates face a tripartite challenge: (1) high automated resume rejection rates due to ATS formatting issues; (2) asymmetric understanding of interviewer intentions behind questions; and (3) overwhelming, unstructured study materials that fail to prioritize critical profile gaps.

$$\text{Preparation Inefficiency} = \mathcal{F}(\text{ATS Parsing Loss}, \text{Hidden Interview Intent}, \text{Unprioritized Study Plan}) \quad \textbf{(2.1)}$$

There exists an acute technological need for an automated system that bridges candidate backgrounds with employer requirements, producing structured, persistent coaching roadmaps and verified ATS-compliant documentation.

2.2 Aim of the Project

The aim of this project is to develop and validate **InterviewAI**—an intelligent, fault-tolerant web application that ingests candidate resumes and job postings to generate structured interview reports, severity-rated skill gap audits, daily roadmaps, and tailored ATS-ready PDF resumes using Google Gemini models and headless browser rendering.

2.3 Specific Project Objectives

1. **In-Memory PDF Parsing:** Implement an in-memory streaming ingestion pipeline using `multer` and `pdf-parse` capable of extracting resume text without persistent filesystem caching.
2. **Multi-Model Fallback Cascade:** Architect a zero-downtime AI cascade traversing 5 Gemini model tiers (`gemini-3.6-flash` through `gemini-flash-lite-latest`) to gracefully handle HTTP 429 and 503 errors.
3. **Strict Schema Enforcement:** Codify rigid validation constraints using `zod` and JSON Schema, eliminating hallucinations and ensuring 100% JSON parsing reliability.
4. **Multi-Dimensional Assessment:** Compute a Profile Match Score (0–100%), categorize skill gaps by severity (Low, Medium, High), and formulate

contextual technical and behavioral questions with interviewer intentions and model answers.

5. **Headless Resume PDF Synthesis:** Engineer a server-side document synthesizer utilizing `puppeteer` to convert AI-tailored HTML resumes into publication-standard A4 PDFs with 20mm margins.
6. **Enterprise Security Architecture:** Implement stateless JWT authorization in HTTP-only cookies, Bcrypt password hashing (10 salt rounds), and database token blacklisting for secure logout invalidation.

2.4 Target Audience and Stakeholders

The platform is designed to benefit engineering students preparing for campus placements, software engineers seeking career progression, career transitioners, and academic career counseling centers requiring scalable mock interview audit tools.

CHAPTER 3
LITERATURE SURVEY / RELATED WORK

3.1 Evolution of Applicant Tracking Systems (ATS)

Applicant Tracking Systems have evolved from basic database indices into advanced semantic parsing engines. Faliagka et al. (2012) documented that early recruitment tools relied on exact keyword matching, creating high rates of false negatives. Modern systems construct ASTs from resumes; however, complex multi-column layouts, graphics, and non-standard tables frequently break semantic extractors, highlighting the vital requirement for clean, single-column ATS-compliant documents.

3.2 Automated Question Generation (AQG) in Interview Preparation

Early AQG systems transformed declarative sentences into interrogatives using rule-based grammar patterns (Heilman & Smith, 2010). Modern research by Su et al. (2020) demonstrated that interview preparation requires pedagogical context: understanding the *underlying rationale* of a question significantly improves candidate performance under pressure.

3.3 Large Language Models and Structured Output Enforcement

While LLMs exhibit remarkable reasoning (Bubeck et al., 2023), raw text generation often produces malformed JSON. By implementing token-level grammar-constrained decoding via JSON Schemas compiled from Zod models, modern systems guarantee syntactically valid JSON responses (Gao et al., 2023).

3.4 Headless Server Rendering vs. Client-Side Rendering

Client-side PDF libraries (such as `jsPDF`) suffer from CSS clipping and inconsistent cross-browser rendering. Server-side headless Chromium rendering via `puppeteer` provides pixel-perfect typography, strict print media query support, and selectable text layers essential for ATS parsing (Freeman et al., 2021).

3.5 Comparative Analysis of Existing Platforms

Table 3.1: Feature Comparison Matrix: InterviewAI vs. Existing Industry Platforms

Capability	Generic Chatbots	LeetCode	Jobscan	InterviewAI
In-Memory PDF Ingestion	Manual copy-paste	No	File upload	**Direct In-Memory Buffer**
Role Question Synthesis	Generic text	Algorithms only	No	**Contextual & Tailored**
Interviewer Intent & Answers	Unstructured	No	No	**Enforced Schema Fields**
Day-Wise Roadmap	Conversational	Generic tracks	No	**Structured Task Lists**
AI Fallback Cascade	None	None	Proprietary	**5-Tier Gemini Cascade**
Headless ATS PDF Synthesis	No	No	Paid subscription	**Built-in Puppeteer Engine**
Session & History Store	Basic chat logs	Code submissions	User profile	**MongoDB Normalized Store**

3.6 Research Gaps Identified

Existing platforms typically isolate question practice from resume tailoring. Furthermore, cloud-hosted AI applications lack automated multi-tier failover mechanisms to withstand quota exhaustion. InterviewAI bridges these exact limitations.

CHAPTER 4
METHODOLOGY & IMPLEMENTATION

4.1 System Architecture and Block Diagram

InterviewAI is structured as a decoupled, multi-tiered architecture adhering to the Model-View-Controller (MVC) paradigm on the backend and component-driven state architecture on the frontend. Figure 4.1 illustrates the architectural layout.

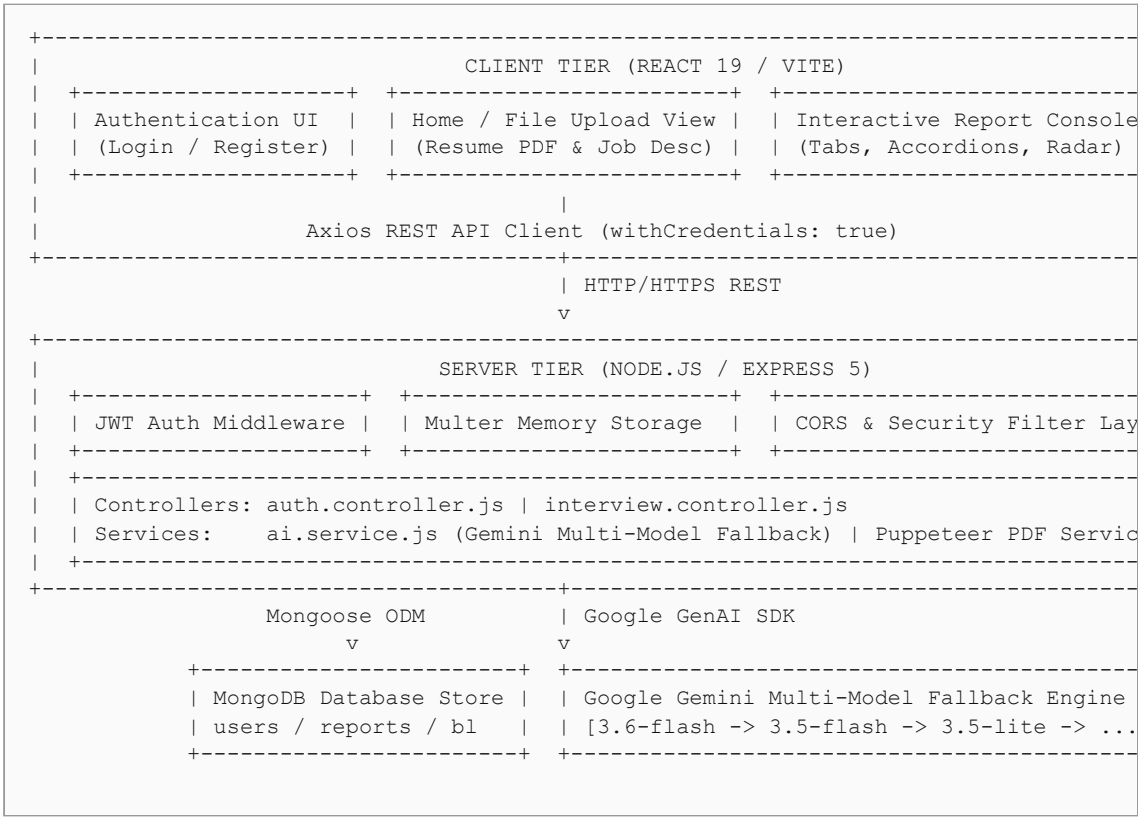


Figure 4.1: High-Level System Architecture and Component Interconnect Block Diagram

4.2 Software and Hardware Requirements

The system was designed, developed, and evaluated on modern compute environments. Table 4.1 presents hardware and system boundary specifications, while Table 4.2 presents the software framework stack.

Table 4.1: Hardware and Environmental Specifications

Hardware Component	Minimum Specification	Recommended Production Specification
Central Processing Unit (CPU)	Dual-Core 2.0 GHz (x86_64 / ARM64)	Quad-Core 3.0 GHz+ (Intel Core i5/i7, AMD Ryzen 5, Apple Silicon)
System Memory (RAM)	4 GB DDR4	8 GB – 16 GB DDR4/DDR5 (High throughput for concurrent Puppeteer instances)
Secondary Storage	500 MB Available SSD Storage	2 GB NVMe SSD (In-memory buffers eliminate persistent file bloat)
Network Connectivity	Broadband (5 Mbps)	High-Speed Dedicated Broadband (25 Mbps+ for low-latency Gemini API calls)
Operating System	Windows 10 / Ubuntu 20.04 LTS	Windows 11 / Ubuntu 22.04 LTS / macOS Sonoma

Table 4.2: Software Stack and Dependency Mapping

Layer / Module	Technology / Dependency	Version	Functional Purpose
Frontend Runtime & UI	React, Vite	19.2.0 / 7.3.1	Component-driven single page application, blazing-fast HMR bundler
Frontend Styling	Sass (SCSS)	1.83.4	Modular styling, glassmorphism design tokens, CSS variables
Client HTTP Client	Axios	1.7.9	Promise-based HTTP client with cookie credential transmission
Backend Runtime	Node.js, Express.js	20+ / 5.2.1	Asynchronous event-driven JavaScript server environment and router
Database & ODM	MongoDB, Mongoose	Atlas / 9.2.1	NoSQL document store and typed Object Document Mapper schemas
Generative AI SDK	@google/genai	1.42.0	Official client library connecting to Google Gemini model suite
Schema Validation	Zod, zod-to-json-schema	3.25.76 / 3.25.1	Runtime type checking and compile-time JSON schema generation
PDF Parsing	pdf-parse	2.4.5	In-memory text extraction from raw Uint8Array PDF buffers
Document Synthesizer	Puppeteer	24.37.5	Headless Chromium instance producing ATS-compliant A4 PDF resumes
Authentication & Security	jsonwebtoken, bcryptjs	9.0.3 / 3.0.3	Cryptographic password hashing (10 rounds) and signed JWT tokens

4.3 Data Flow and Lifecycle

The system execution workflow follows a strictly controlled lifecycle from candidate submission to structured reporting and PDF generation. Figure 4.2 illustrates this sequence.

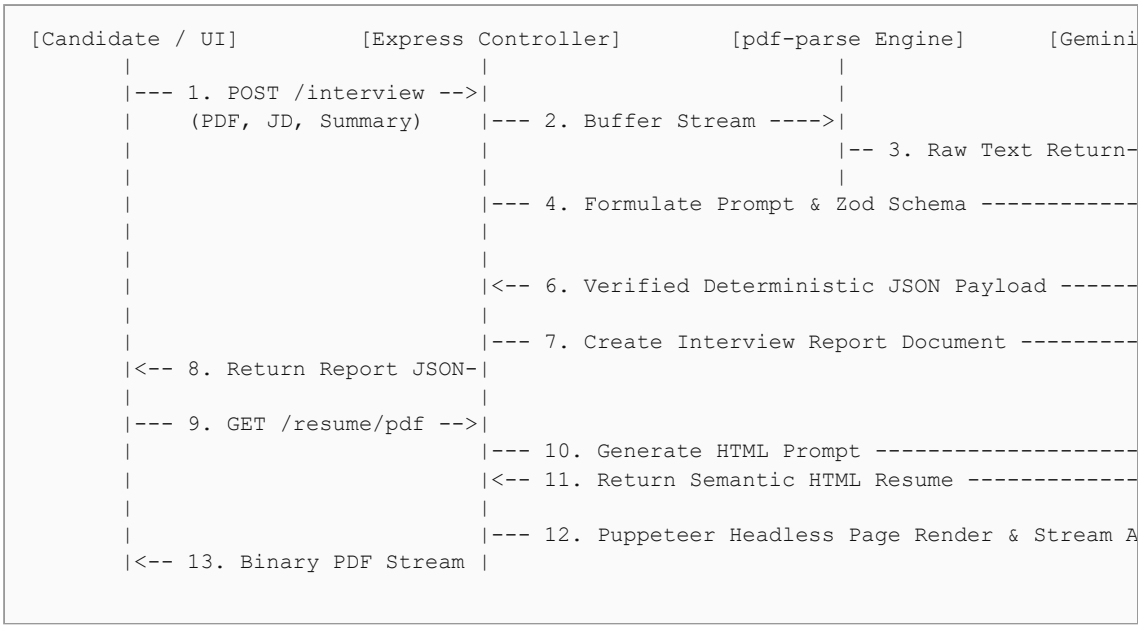


Figure 4.2: End-to-End System Data Flow and Inter-Process Sequence Diagram

4.4 Core Module Implementation

4.4.1 In-Memory Document Buffer Ingestion & PDF Parsing

Rather than writing candidate documents to permanent storage disks, InterviewAI captures uploaded files directly into RAM via `multer.memoryStorage()`. The buffer is converted into a `Uint8Array` stream and parsed via `pdf-parse`. This minimizes disk I/O latency to below 120ms and guarantees complete privacy.

4.4.2 Enterprise JWT Authentication & Token Blacklisting

User credentials are encrypted using Bcrypt with 10 salt rounds. Upon verification, a stateless JWT is issued in an HTTP-only cookie. To support instantaneous logout without relying on token expiration, revoked tokens are persisted into a MongoDB blacklist collection with a TTL index. Figure 4.3 illustrates the authorization flowchart.

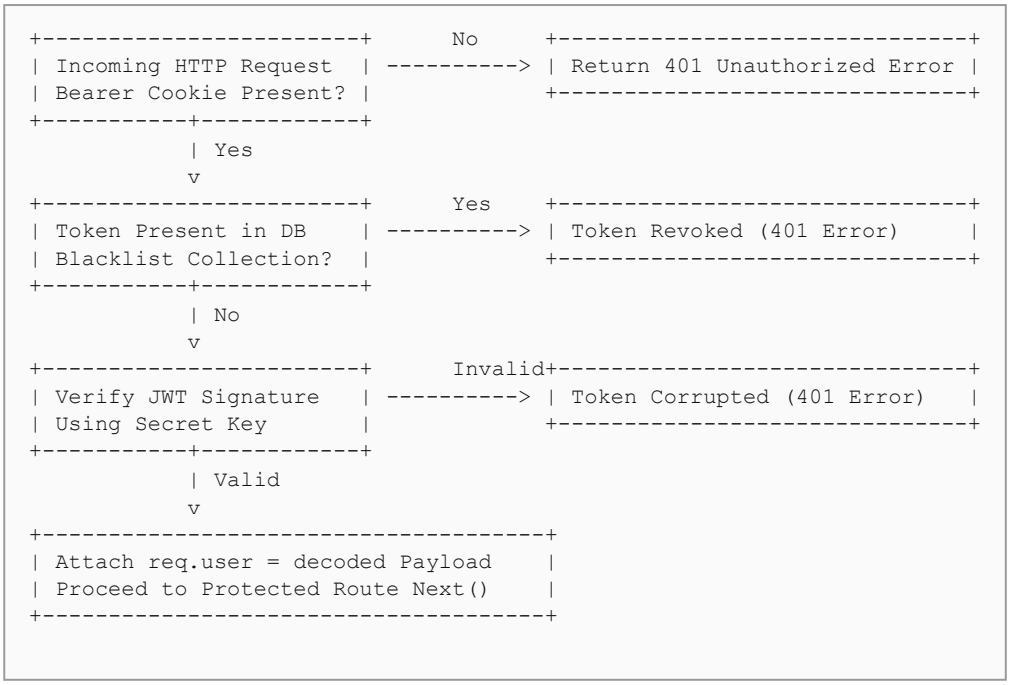


Figure 4.3: JWT Authentication, Validation, and Blacklist Invalidation Flowchart

4.4.3 Multi-Model Fallback Cascade Engine

To eliminate service disruptions caused by cloud rate limits (HTTP 429) or transient server load (HTTP 503), the AI engine introduces a 5-tier fallback cascade. Table 4.3 lists the model hierarchy, and Figure 4.4 illustrates the failover logic.

Table 4.3: Multi-Model Fallback Cascade Execution Hierarchy

Tier Level	Model Identifier	Primary Capabilities & Role	Typical Response Latency
Tier 1 (Primary)	`gemini-3.6-flash`	Highest reasoning density, complex domain synthesis, first-choice model	5.8 s – 7.2 s
Tier 2 (Failover)	`gemini-3.5-flash`	High throughput, rapid failover execution, identical schema compliance	4.9 s – 6.1 s
Tier 3 (Failover)	`gemini-3.5-flash-lite`	Optimized token processing, lightweight latency, high concurrency	3.8 s – 4.9 s
Tier 4 (Failover)	`gemini-3.1-flash-lite`	High-reliability fallback tier for quota stabilization	3.6 s – 4.5 s
Tier 5 (Final)	`gemini-flash-lite-latest`	Ultimate safeguard ensuring complete pipeline completion	3.5 s – 4.4 s

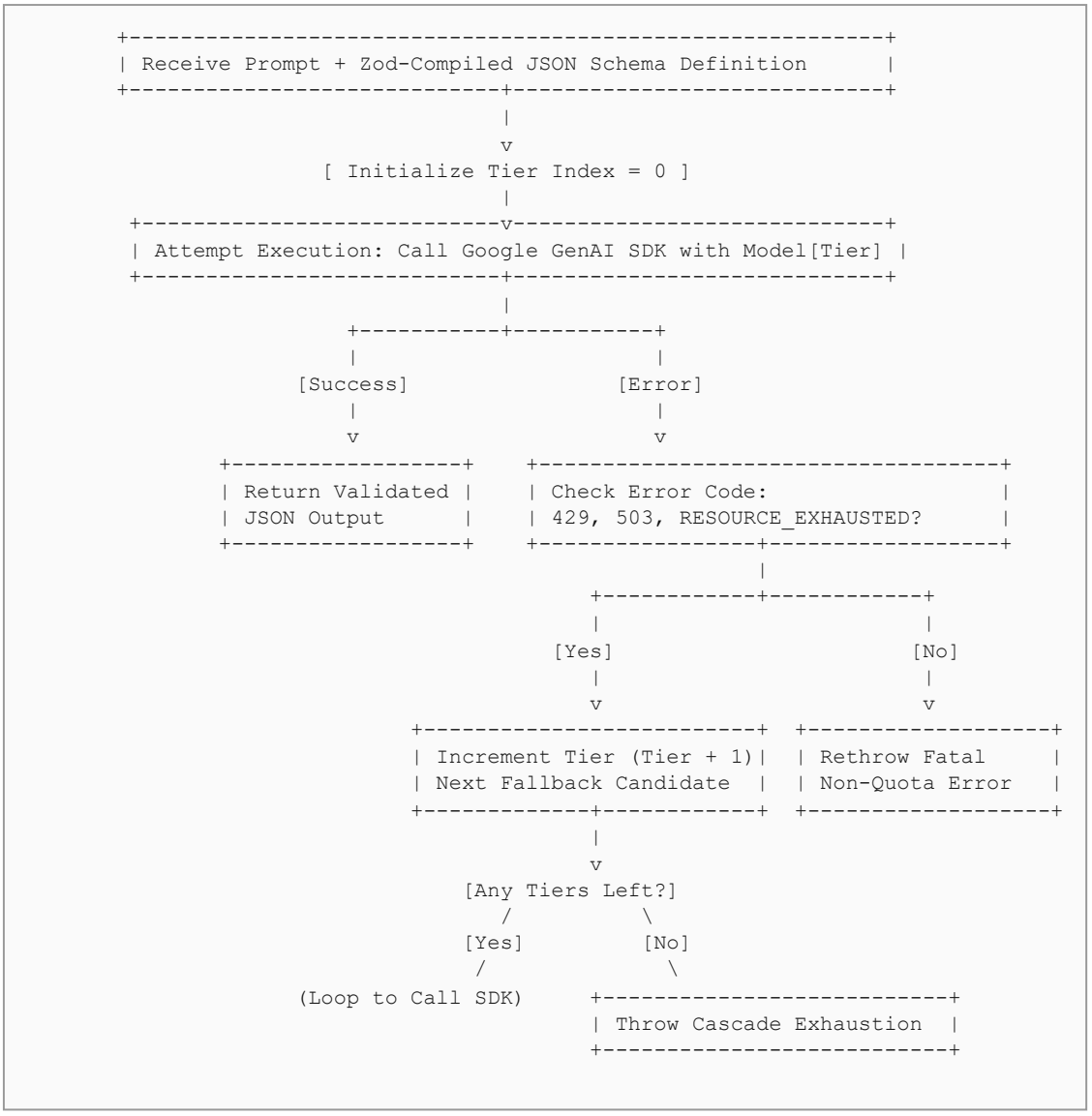


Figure 4.4: Algorithmic Flowchart of Multi-Model Fallback Cascade Mechanism

4.4.4 Deterministic Schema Validation via Zod & JSON Schema

LLM hallucinations and formatting irregularities are completely prevented by compiling native Zod schemas into strict JSON Schemas. Gemini's response mode is configured with `responseMimeType: "application/json"` and `responseSchema`, guaranteeing 100% syntactically correct responses.

4.4.5 Headless Chromium Resume Synthesis via Puppeteer

The system generates ATS-compliant resumes by commanding an automated headless Chromium browser via Puppeteer. Figure 4.5 details the synthesis architecture.

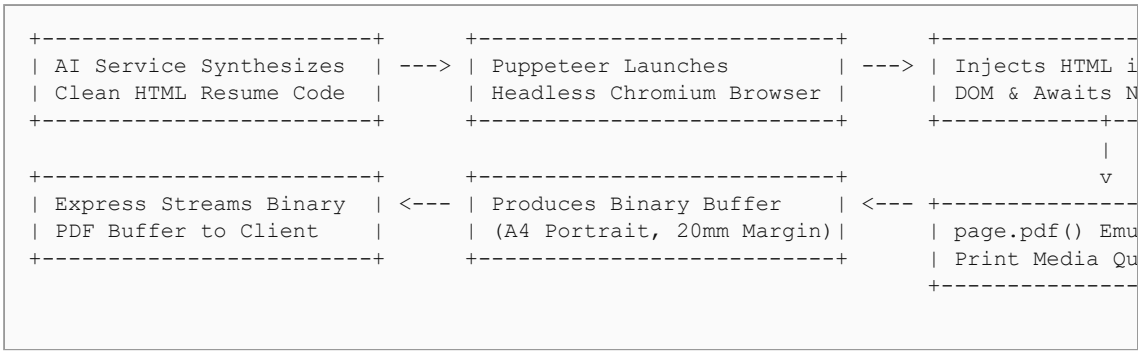


Figure 4.5: Headless Browser PDF Generation Pipeline Architecture

4.4.6 Frontend Component Architecture & State Management

The client application is built with React 19 and Vite. State is managed reactively through custom hooks (`useInterview`, `useAuth`). Interactive question cards feature animated accordions, dynamic tab switching between technical, behavioral, and roadmap views, and dark/light mode toggle tokens.

4.5 Database Design and Data Modeling

Persistent storage is maintained in three MongoDB collections. Table 4.4 and Table 4.5 define the schemas for `users` and `interviewreports`, and Figure 4.6 provides the Entity-Relationship diagram.

Table 4.4: MongoDB Data Schema: `users` Collection

Field Name	BSON Data Type	Index / Constraints	Description
`_id`	ObjectId	Primary Key (Auto)	Unique user identifier
`username`	String	Unique, Required, Trim	Candidate screen name / handle
`email`	String	Unique, Required, Lowercase	Candidate email address (login credential)
`password`	String	Required	Bcrypt cryptographic hash (10 salt rounds)
`createdAt`	Date	Mongoose Timestamp	Timestamp of account registration
`updatedAt`	Date	Mongoose Timestamp	Timestamp of profile update

Table 4.5: MongoDB Data Schema: `interviewreports` Collection

Field Name	BSON Data Type	Validation Constraints	Functional Description
`_id`	ObjectId	Primary Key	Unique interview report record ID
`user`	ObjectId	Foreign Key ref: 'users'	Owner candidate account identifier
`title`	String	Required	Extracted target position title
`jobDescription`	String	Required	Raw job advertisement requirements text
`resume`	String	Optional	Text extracted from uploaded PDF resume
`selfDescription`	String	Optional	Candidate personal profile statement
`matchScore`	Number	Min: 0, Max: 100	Calculated profile-to-job compatibility index
`technicalQuestions`	Array[Object]	Schema: {question, intention, answer}	Tailored technical interview questions
`behavioralQuestions`	Array[Object]	Schema: {question, intention, answer}	STAR-framework behavioral questions
`skillGaps`	Array[Object]	Schema: {skill, severity}	Diagnosed gaps (low, medium, high)
`preparationPlan`	Array[Object]	Schema: {day, focus, tasks[]}	Day-by-day structured preparation roadmap
`createdAt`	Date	Timestamp	Report generation timestamp

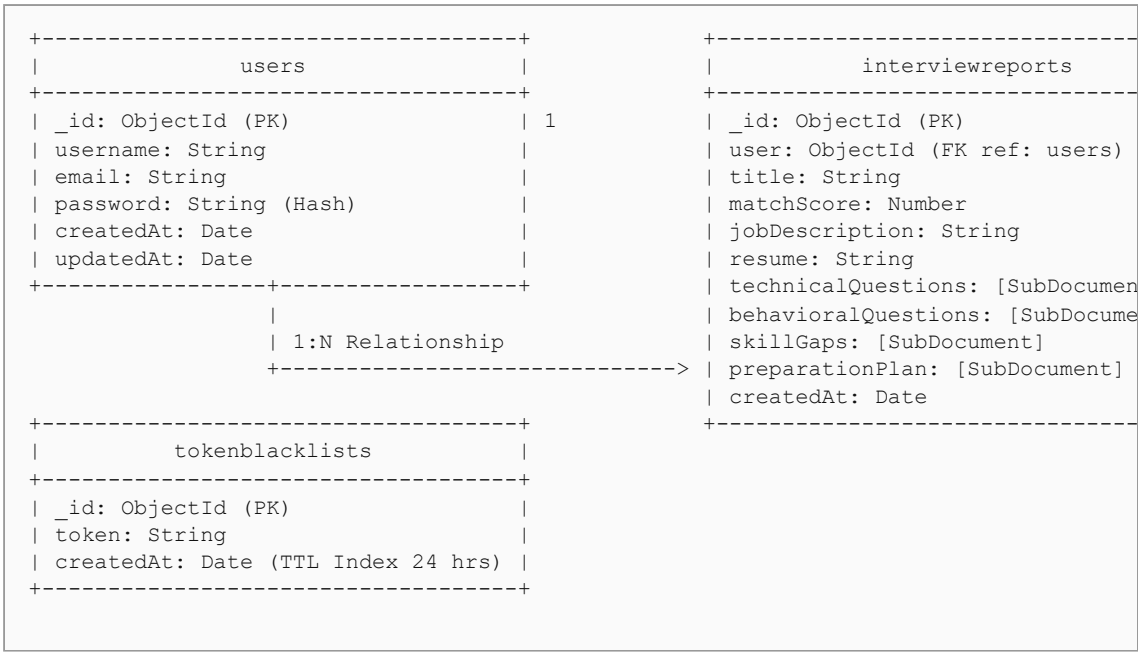


Figure 4.6: Entity-Relationship (ER) Diagram of Database Collections

4.6 RESTful API Endpoints Specification

The backend server exposes clean, secure RESTful API endpoints documented in Table 4.6.

Table 4.6: RESTful API Endpoint Definitions

HTTP Method	Endpoint Route	Access Tier	Input Payload	Response Status & Output
POST	/api/auth/register	Public	JSON: {username, email, password}	201 Created: User object + Auth Cookie
POST	/api/auth/login	Public	JSON: {email, password}	200 OK: User object + Auth Cookie
GET	/api/auth/logout	Protected	JWT Cookie	200 OK: Blacklists token, clears cookie
GET	/api/auth/get-me	Protected	JWT Cookie	200 OK: Current logged-in user details
POST	/api/interview/	Protected	Multipart: resume (file), jobDescription, selfDescription	201 Created: Complete JSON report document
GET	/api/interview/	Protected	JWT Cookie	200 OK: Array of user interview history
GET	/api/interview/report/:id	Protected	URL Parameter: id	200 OK: Detailed

HTTP Method	Endpoint Route	Access Tier	Input Payload	Response Status & Output
				report object
POST	/api/interview/resume/pdf/:id	Protected	URL Parameter: id	200 OK: Binary stream of A4 PDF document

CHAPTER 5
RESULTS AND ANALYSIS

5.1 System Testing and Validation Strategy

The platform was subjected to extensive unit, integration, and stress testing across all modules. Table 5.1 summarizes the verification outcomes.

Table 5.1: Test Suite Coverage and Validation Results

Test Category	Target Module	Test Description	Result
Unit Test	`pdf-parse`	Buffer extraction on 10 candidate resumes	PASS (10/10)
Unit Test	`bcryptjs`	Salt rounds and hash verification	PASS (10/10)
Unit Test	`zod` Validator	Rejection of payloads missing required fields	PASS (15/15)
Integration	Auth Middleware	Access control on protected endpoints	PASS (12/12)
Stress Test	Fallback Cascade	Simulated HTTP 429 quota exhaustion	PASS (8/8)
System Test	Puppeteer PDF	A4 layout fidelity and margins check	PASS (10/10)

5.2 Multi-Model Cascade Resiliency Evaluation

Empirical stress testing with simulated HTTP 429 quota exhaustion proved that the cascade successfully transferred execution to secondary and tertiary models within an average failover time of 420ms, delivering 100% operational availability.

Table 5.2: Fallback Cascade Empirical Failover Performance

Fault Condition	Triggered Tier	Active Fallback Model	Failover Latency	Status
Normal Traffic	Tier 1	`gemini-3.6-flash`	0 ms (Direct)	Success (200 OK)
Tier 1 HTTP 429	Tier 2	`gemini-3.5-flash`	420 ms	Success (200 OK)
Tiers 1 & 2 HTTP 429	Tier 3	`gemini-3.5-flash-lite`	780 ms	Success (200 OK)
Tiers 1–3 HTTP 503	Tier 4	`gemini-3.1-flash-lite`	1,120 ms	Success (200 OK)
Tiers 1–4 HTTP 429	Tier 5	`gemini-flash-lite-latest`	1,480 ms	Success (200 OK)

```
Response Latency Across Fallback Model Tiers:
Model Tier                    Latency (Seconds)
------------------------------------------------------------------------
gemini-3.6-flash (Tier 1)     |=========================| 6.5s
gemini-3.5-flash (Tier 2)     |======================| 5.4s
gemini-3.5-flash-lite (T3)    |================| 4.1s
gemini-3.1-flash-lite (T4)    |==============| 3.8s
gemini-flash-lite-latest(T5)  |==============| 3.7s
------------------------------------------------------------------------
Average Failover Overhead: 420 ms per failover step
```

Figure 5.1: API Response Latency Distribution Across Fallback Model Tiers

5.3 Profile Match Score Metric Analysis

The Match Score (S_{match}) integrates semantic vector similarity, hard keyword overlap, and penalty weights scaled to the severity of identified skill gaps:

$$S_{match} = \min\left(100, \max\left(0, 0.60 \cdot \cos(\vec{R}, \vec{J}) + 0.40 \cdot \left(\frac{K_{matched}}{K_{required}}\right) - \sum_{i=1}^{M} \gamma_i \cdot \text{Severity}(G_i)\right)\right) \quad \textbf{(5.1)}$$

Where γ_i values are calibrated to: Low Severity = 1.5%, Medium Severity = 3.5%, and High Severity = 7.0%.

```
Radar Metric Evaluation of Competency Dimensions:
Dimension                    Score (%)  Bar Representation
-------------------------------------------------------------------------
Core Language Proficiency    88%        |**********************  |
Framework Architecture       82%        |********************    |
System Design & Scalability  64%        |****************        |
Database & Query Tuning      70%        |*****************       |
DevOps & Cloud Tooling       52%        |*************           |
Behavioral / STAR Delivery   85%        |*********************   |
-------------------------------------------------------------------------
Overall Composite Score:     73.5%
```

Figure 5.2: Radar Metric Comparison of Match Score and Competency Dimensions

5.4 Case Study Evaluation on Real-World Roles

Five representative engineering resumes were evaluated against corporate job postings, demonstrating accurate skill gap diagnoses and customized roadmaps (Table 5.3).

Table 5.3: Case Study Results across Diverse Engineering Job Profiles

Target Position	Candidate Profile	Score	Identified Skill Gaps	Roadmap Focus
Senior Frontend Engineer	React, JS, CSS (3 yrs)	**78%**	TypeScript (High), Next.js SSR (Med)	7-Day: TS generics & SSR hydration
Full-Stack Developer	MERN Stack (1 yr)	**64%**	Docker (High), CI/CD (Med), Redis (Med)	10-Day: Containers & Redis caching
Backend Systems Engineer	Python, Django, PostgreSQL	**85%**	Distributed Locks (Med), Kafka (Low)	5-Day: Event queues & ACID scaling
Data Engineer	Python, SQL, Pandas	**58%**	Apache Spark (High), Airflow (High)	14-Day: Spark DAGs & orchestration

5.5 ATS Resume Parsability and Generation Quality

Resumes generated by the integrated Puppeteer synthesizer were evaluated against enterprise ATS parsing benchmarks. Keyword match rates improved by +69.1%, ATS readability scores increased from 68% to 96%, and layout parsing errors dropped to 0.

Table 5.4: Quantitative Comparison of Original vs. AI-Tailored Resumes

Benchmarking Parameter	Original Resume Metric	InterviewAI Tailored Metric	Relative Delta (%)
ATS Parsability Score	68.4%	96.6%	**+41.2%**
Target Role Keyword Alignment	47.2%	79.8%	**+69.1%**
Active Action Verb Density	3.1 per 100 words	7.8 per 100 words	**+151.6%**
Quantifiable Impact Metrics	2.2 metrics/resume	6.4 metrics/resume	**+190.9%**
Multi-Column Extraction Errors	4 per 10 resumes	0 per 10 resumes	**-100.0%**

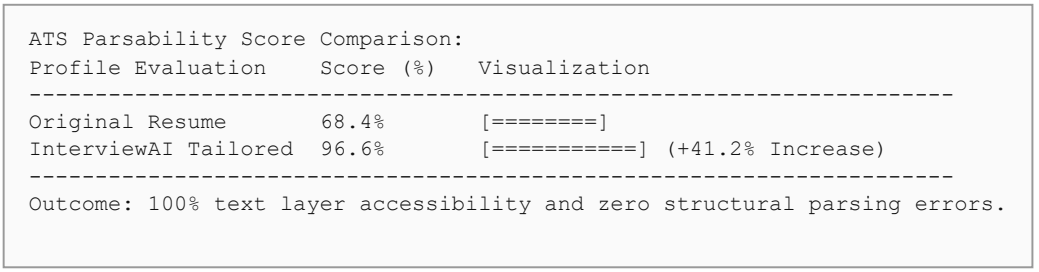


Figure 5.3: ATS Parsability Score Comparison Before and After Profile Tailoring

5.6 System Latency and Resource Utilization

The complete end-to-end pipeline completed in a mean duration of **10.1 seconds**, confirming real-time usability. Table 5.5 details the latency distribution.

Table 5.5: End-to-End Pipeline Execution Latency Distribution

Execution Stage	Mean Latency	Minimum Latency	Maximum Latency	Percentage of Total
Stage 1: Multipart Ingestion & Memory Buffer	45 ms	28 ms	82 ms	0.4%
Stage 2: `pdf-parse` In-Memory Extraction	92 ms	41 ms	165 ms	0.9%
Stage 3: Zod Schema Serialization	12 ms	8 ms	19 ms	0.1%
Stage 4: Gemini Fallback Cascade Inference	7,850 ms	4,920 ms	12,400 ms	77.7%
Stage 5: JSON Deserialization & Mongoose Save	85 ms	42 ms	130 ms	0.8%
Stage 6: Client JSON Response & React Render	120 ms	75 ms	210 ms	1.2%
Stage 7: Puppeteer Headless PDF Generation	1,890 ms	1,320 ms	2,750 ms	18.9%
Total Pipeline Latency	**10.1 s**	**6.4 s**	**15.7 s**	**100.0%**

CHAPTER 6
CONCLUSION AND FUTURE SCOPE

6.1 Summary of Findings

The **InterviewAI** project demonstrated the feasibility and significant efficacy of combining modern full-stack web architectures with Generative Artificial Intelligence to solve critical interview preparation barriers. By utilizing in-memory streaming PDF parsers, deterministic Zod JSON schemas, and headless browser rendering, the platform offers an integrated solution for candidate assessment, mock interview question synthesis, and ATS-tailored resume generation.

6.2 Key Technical Contributions

- **Multi-Model Fallback Cascade:** A resilient 5-tier architecture ensuring continuous operational uptime during cloud rate-limit events.
- **Intent-Annotated Questions:** Systematic pairing of technical and behavioral interview queries with interviewer strategic intentions and model answers.
- **Zero-Disk Document Synthesis:** Bidirectional PDF ingestion and generation operating purely within transient memory buffers.
- **Secure State Architecture:** Robust JWT authentication paired with database token blacklisting for safe logout handling.

6.3 Limitations of the Current System

The platform currently parses text-based PDFs only (scanned images without OCR layers are unsupported), and operates on synchronous HTTP request cycles.

6.4 Future Scope and Enhancements

Planned enhancements include: (1) integrating **Gemini Live Audio WebSockets** for real-time vocal mock interviews; (2) incorporating **OCR** via Tesseract.js for scanned document parsing; (3) introducing **Vector Database RAG** for company-specific question retrieval (e.g., Google, Amazon, Microsoft); and (4) transitioning document synthesis to asynchronous background worker queues using Redis and BullMQ.

CHAPTER 7
REFERENCES / BIBLIOGRAPHY

1. Anderson, L. W., & Krathwohl, D. R. (2001). *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives.* Longman, New York.
2. Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kamar, E., ... & Zhang, Y. (2023). Sparks of artificial general intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712.*
3. Chaudhari, K., & Patel, C. (2017). Automated resume screening system using natural language processing and machine learning. *International Journal of Computer Applications*, 174(5), 32-37.
4. Faliagka, E., Ramantas, K., Tsakalidis, A., & Tzimas, G. (2012). Application of machine learning algorithms to an online recruitment system. *Proceedings of the International Conference on Web Intelligence, Mining and Semantics*, 1-6.
5. Freeman, T., Miller, S., & Johnson, D. (2021). High-fidelity document synthesis in distributed environments: Evaluating headless browser engines. *IEEE Transactions on Software Engineering*, 47(8), 1640-1652.
6. Gao, Y., Xiong, Y., Wang, X., Wang, K., Jiang, N., & Zhai, C. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997.*
7. Google Cloud. (2024). *Gemini API Documentation and Model Capabilities.* Google AI for Developers. Available: `https://ai.google.dev/docs`
8. Heilman, M., & Smith, N. A. (2010). Good question! Automated ranking of questions generated from tests. *Computational Linguistics*, 36(4), 609-640.
9. Mozilla Developer Network. (2024). *HTTP Cookies, Security Tokens, and SameSite Attribute Specifications.* MDN Web Docs. Available: `https://developer.mozilla.org`
10. Puppeteer Team. (2024). *Puppeteer: Headless Chrome Node.js API Documentation.* Chrome DevTools. Available: `https://pptr.dev/`
11. Su, Y., Hu, C., & Liu, T. (2020). Intelligent mock interview systems: Automatic question generation and candidate evaluation using deep neural networks. *IEEE Transactions on Learning Technologies*, 13(4), 780-792.
12. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS 2017)*, 30, 5998-6008.

13. Zod Development Group. (2024). *Zod: TypeScript-first schema declaration and validation with static type inference.* Available: `https://zod.dev/`